

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВИЙ ПРОЄКТ
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ПРОДУКТОВИЙ
МАГАЗИН».

Студентки 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Недєвої Дарини В'ячеславівни

Керівник: к.т.н., доц. Антоненко О.С.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.

(підпис) (прізвище та ініціали)

_____ Шестопапов С.В.

(підпис) (прізвище та ініціали)

(підпис) (прізвище та ініціали)

ОДЕСА - 2026

ЗМІСТ

	стор.
ВСТУП	3
ЗАВДАННЯ	4
1. ТЕОРЕТИЧНА ЧАСТИНА	5
1.1 Поняття ООП	5
1.2 Опис об'єктної частини.....	6
2. ПРАКТИЧНА ЧАСТИНА	7
2.1 Опис класів	7
2.3 Інтерфейс.....	10
ВИСНОВКИ.....	24
СПИСОК ЛІТЕРАТУРИ.....	25

ВСТУП

Мета роботи – створити ієрархію класів на тему «Гіпермаркет» та її застосуванні для вирішення прикладної задачі, визначеної у варіанті завдання. У ході виконання роботи необхідно реалізувати концепції об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Згідно з вимогами завданням, слід розробити систему взаємопов'язаних класів, яка описуватиме структуру і функціонування продуктового магазину.

Для цього потрібно створити конструктори класів, визначити поля та реалізувати методи для роботи з об'єктами. Поля класів мають бути закритими, а доступ до них забезпечуватиметься через відповідні методи класу. У програмі необхідно побудувати ієрархію класів, базуючись на відносинах успадкування та композиції. Для реалізації динамічного поліморфізму слід використовувати віртуальні методи, класи, а за потреби - чисті віртуальні методи.

ЗАВДАННЯ

Варіант 20. Створення ієрархії класів на тему «Продуктовий магазин».

Створити класи: «Товар» (найменування, вага, дата випуску), «Продукт харчування» (спадкоємець класу Товар, з полем – термін придатності, кількість калорій, метод - чи закінчився термін придатності), «Йогурт» (спадкоємець класу Продукт харчування, додатково - жирність, начинка), «Молоко» (спадкоємець класу Продукту харчування, вид упаковки, жирність) та ще один вид товарів на вибір студента. Також реалізувати клас «Полиця» (місткість: кількість товарів, що зберігає товари).

В основному проекті передбачити можливість постачання товарів (задається товар та його кількість), розміщення товарів по полицях (алгоритм розміщення на розсуд студента), списання прострочених товарів, купівля товару. За бажанням студента додати поле ціна в товар, та додатково підраховувати виручку магазину (тобто загальну ціну всіх проданих товарів).

1.ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтоване програмування – це концепсія, в межах якої програма розглядається як сукупність об'єктів, що обмінюються повідомленнями [1]. Кожен об'єкт є екземпляром певного класу, а класи, своєю чергою, утворюють ієрархію на основі наслідування. Розробник спочатку визначає класи (шаблони), а під час виконання програми на їхній основі створюються конкретні об'єкти. Нові класи можуть успадковувати властивості від базових, що й породжує ієрархічну структуру.

Будь-який стиль програмування має свою концептуальну базу [2]. Для об'єктно-орієнтованого стилю такою базою є об'єктна модель, яка ґрунтується на трьох основних принципах [3]:

- а) інкапсуляція;
- б) поліморфізм;
- с) наслідування.

Інкапсуляція – механізм, який поєднує в одному об'єкті даних і методів їхньої обробки, а також приховування внутрішньої реалізації від зовнішнього світу, що запобігає помилковому чи несанкціонованому втручанню [2].

Наслідування – процес, завдяки якому один об'єкт (або клас) може набути властивостей іншого, доповнюючи їх власними, унікальними характеристиками. Це дозволяє будувати ієрархії класів від загального до конкретного [2].

Поліморфізм – здатність одного й того самого імені (наприклад, назви методу) позначати різні дії залежно від типу об'єкта, якому адресоване

повідомлення [2]. На практиці це означає, що об'єкт сам обирає відповідний внутрішній метод відповідно до отриманих даних. Головна мета поліморфізму – забезпечити єдиний інтерфейс для класу споріднених операцій.

1.2 Опис об'єктної частини

Програма реалізує консольний застосунок для обліку харчових товарів на полиці магазину. Система підтримує три категорії продуктів: йогурт, молоко та хліб. Кожен тип має власний набір характеристик, але всі вони успадковують спільні властивості від базового класу `Product` через проміжний клас `ProductFood`.

Магазин представлений полицею (`Shelf`), яка характеризується обмеженою місткістю (кількістю вільних місць для товарів). Полиця містить список екземплярів харчових продуктів. Новий товар може бути доданий лише в тому випадку, якщо на полиці є вільне місце; при спробі додати більшу кількість товарів, ніж дозволяє місткість, операція видає помилку.

Користувач взаємодіє із системою через консольне меню класу `UserInterface`, де може додавати товари обраного типу, переглядати асортимент на полиці, перевіряти термін придатності, сортувати товари за ціною, оформлювати покупку, переглядати виручку, перевіряти цілісність товарів та змінювати місткість полиці. При введенні некоректних даних (наприклад, від'ємна вага чи ціна, некоректні дати, перевищення допустимої жирності) програма виводить повідомлення про помилку за допомогою механізму винятків і продовжує роботу.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Діаграму класів наведено в додатку 1. Розглянемо більш детально структуру кожного класу.

1) Базовий клас «Product»:

a) Приватні поля: name, produceDate, weight, price.

b) Конструктори:

- Конструктор без параметрів;
- Конструктор з параметрами.

c) Методи:

- Методи доступу getName, getProduceDate, getWeight, getPrice – повертають значення відповідних полів;
- Info – віртуальний метод, повертає текстовий опис товару (назва, дата виробництва, вага, ціна).

2) Клас «ProductFood», нащадок класу «Product»:

a) Приватні поля: shelfLife, amountCalories.

b) Конструктори:

- Конструктор без параметрів;
- Конструктор з параметрами.

c) Методи:

- Методи доступу getShelfLife, getAmountCalories – повертають термін придатності та кількості калорій;
- Статичний метод convertDate – перетворює дати формату ДД/ММ/РРРР у числове значення;

- Статичний метод `dateComparison` – метод перевірки, що дата виробництва не пізніша за термін придатності;
- `isShelfLifeGood` – перевіряє, чи не прострочений товар відносно поточної дати;
- Перевизначення `Info` – повертає розширений опис харчового продукту.

3) Клас «Yogurt», нащадок класу «ProductFood»:

a) Приватні поля: `fat`, `inside`.

b) Конструктори:

- Конструктор без параметрів;
- Конструктор з параметрами.

c) Методи:

- `getFat`, `getInside` – повертають жирність та начинку;
- Перевизначення `Info` – повертає інформацію про йогурт.

4) 4. Клас «Milk», нащадок класу «ProductFood»:

a) Приватні поля: `fat`, `packageType`.

b) Конструктори:

- Конструктор без параметрів;
- Конструктор з параметрами.

c) Методи:

- `getFat`, `getPackageType` – повертають жирність молока та тип упаковки;
- `setFat` – встановлює жирність з перевіркою допустимого діапазону (0–4 %);
- Перевизначення `Info` – повертає інформацію про молоко.

5) Клас «Bread», нащадок класу «ProductFood»:

a) Приватні поля: `flourType`, `isWholeGrain`.

b) Конструктори:

- Конструктор без параметрів;

- Конструктор з параметрами.

с) Методи:

- `getFlourType`, `getIsWholeGrain` – повертають тип борошна та ознаку цільнозернового хліба;

- Перевизначення `Info` – повертає інформацію про хліб.

6) Клас «Shelf»:

а) Приватні поля: `capacity`, `damage`, `amountMoney`, `vecAmountProduct`.

б) Конструктори:

- Конструктор без параметрів.

с) Деструктор:

- `Shelf::~~Shelf()`.

д) Методи:

- `getCapacity`, `getAmountMoney`, `getDamageProduct` – повертають місткість полиці, суму виручки та кількість видалених прострочених товарів;

- `sumPrice` – додає ціну проданого товару до загальної виручки;

- `setCapacityIncrease`, `setCapacityReduce` – збільшують або зменшують кількість вільних місць на полиці;

- `addProductFood` – додає товар на полицю;

- `loseShelfLifeProductFood` – видаляє прострочені товари;

- `sortProductPriceAscending`, `sortProductPriceDescending` – сортують товари за ціною;

- `buyProduct`, `buyAllProduct` – оформлюють покупку одного товару або всіх товарів;

- `checkIntegrity`, `checkAllIntegrity` – перевіряють цілісність товару за назвою або всіх товарів на полиці;
- `printProductFood` – виводить перелік товарів на полиці.

7) Клас «UserInterface»

а) Методи:

- `SystemMenu` – відображає головне консольне меню та обробляє вибір користувача (додавання йогурту, молока, хліба; перевірка терміну придатності; сортування; покупка; перегляд виручки; перевірка цілісності; виведення товарів; зміна місткості полиці; вихід із програми).

2.2 Інтерфейс

Під час запуску програми користувач бачить вітальне повідомлення та головне консольне меню.

Він потрапляє на це вікно одразу після запуску застосунку. На екрані відображається привітання WELCOME TO THE STORE, поточна дата магазину та перелік доступних пунктів меню (1–11). Для виконання дії потрібно ввести номер обраного пункту в рядку Your choice:. У програмі передбачено один тип користувача – оператор магазину, який керує полицею з товарами (Рис 2.1).

```

=====
WELCOME TO THE STORE
=====
Select product:
=====
1. Yogurt.
2. Milk.
3. Bread.
=====
Select operation:
=====
4. Expiry date check.
5. Sorting products on the shelf.
6. Buy products
7. Revenue from sold products .
8. Checking the product for integrity.
9. Products on the shelf.
10. Shelf capacity.
11. Exit from the program.
Today's date 27/05/2026
=====
Select any item
and follow the hints. Have a nice day!
=====
Your choice:

```

Рисунок 2.1 – початкове (головне) меню програми

Після вибору пункту 1 програма пропонує ввести параметри йогурту: назву, дату виробництва (ДД/ММ/РРРР), вагу, ціну, термін придатності, калорійність, жирність (0–10 %), начинку, а також кількість одиниць товару. Перед додаванням виводиться кількість вільних місць на полиці (Рис 2.2).

```

Your choice:
1
You chose yogurt, let's add it to the shelf.
Enter yogurt name:
DANONE
Add the production date as (DD/MM/YYYY):
25/05/2026
Enter weight in grams (not less than 0):
125
Enter price $:
1
Enter the expiration date (DD/MM/YYYY):
01/06/2026
Enter the number of calories (kcal) at least 0:
465
Inject the fat content of the yogurt (0% to 10%) :
2
Inject the yogurt filling (strawberry, apple and other fruit):
Enter the inside of the yogurt:
Strawberry
Free places on the shelf: 6.
Product quantity:
2
2 yogurts have been added to the shelf.

```

Рисунок 2.2 – Вікно додавання йогурту.

Якщо запитана кількість перевищує вільні місця, з'являється повідомлення There is not enough space on the shelf, і товари не додаються. При коректних даних на полицю додається потрібна кількість йогуртів, після чого виводиться підтвердження (Рис 2.3).

```

Free places on the shelf: 3.
Product quantity:
4
There is not enough space on the shelf.

```

Рисунок 2.3 – Вікно при перевищенні вільних місць на полиці

Аналогічно пункту 1 користувач вводить назву, дати, вагу, ціну, термін придатності, калорії, жирність молока (0–4 %), тип упаковки та кількість (Рис 2.4).

При перевищенні місткості полиці операція скасовується з відповідним повідомленням (Рис 2.3).

```

Your choice:
2
You have selected milk, we will add it to the shelf.
Enter the name of the milk:
Lactel
Add the production date as (DD/MM/YYYY):
27/05/2026
Enter weight in grams (not less than 0):
950
Enter price $:
3
Enter the expiration date (DD/MM/YYYY):
03/06/2026
Enter the number of calories (kcal) at least 0:
650
Enter the fat content of the milk (from 0% to 4%):
2.5
Enter the type of packaging (Bottle, pure pack, pillow):
Pure pack
Free places on the shelf: 6.
Product quantity:
3
3 milk (Pure pack) were added to the shelf.

```

Рисунок 2.4 – Вікно додавання молока.

Користувач вводить назву, дати, вагу, ціну, термін придатності, калорії, тип борошна, ознаку цільозернового виробу (1 – так, 0 – ні) та кількість буханок (Рис 2.5). Діють ті самі обмеження щодо вільних місць на полиці (Рис 2.3).

```

Your choice:
3
You chose bread, let's add it to the shelf.
Enter the name of the bread:
Odeskiy
Add the production date as (DD/MM/YYYY):
27/05/2026
Enter weight in grams (not less than 0):
460
Enter price $:
1
Enter the expiration date (DD/MM/YYYY):
28/06/2026
Enter the number of calories (kcal) at least 0:
1101
Enter the type of flour (wheat, rye, buckwheat)):
wheat
Enter 1 if whole grain, if not 0.
1
Free places on the shelf: 6.
product quantity:
3
Added 3 bread loaf.

```

Рисунок 2.5 – Вікно додавання хліб

При введенні некоректних даних (наприклад, від’ємна вага (Рис 2.5) чи ціна (Рис 2.6), невірний формат дати (Рис 2.7), від’ємна калорійність (Рис 2.8), жирність поза допустимим діапазоном (Рис 2.8)) програма виводить текст помилки з винятку exception і повертає користувача до головного меню.

```

2
The weight of the bread cannot be negative.
=====
Select product:
=====
1. Yogurt.
2. Milk.
3. Bread.
=====
Select operation:
=====
4. Expiry date check.
5. Sorting products on the shelf.
6. Buy products
7. Revenue from sold products .
8. Checking the product for integrity.
9. Products on the shelf.
10. Shelf capacity.
11. Exit from the program.
Today's date 27/05/2026
=====
Select any item
and follow the hints. Have a nice day!
=====
Your choice:

```

Рисунок 2.5 – Вікно при від’ємній вагі

```

The price of bread cannot be negative.
=====
Select product:
=====
1. Yogurt.
2. Milk.
3. Bread.
=====
Select operation:
=====
4. Expiry date check.
5. Sorting products on the shelf.
6. Buy products
7. Revenue from sold products .
8. Checking the product for integrity.
9. Products on the shelf.
10. Shelf capacity.
11. Exit from the program.
Today's date 27/05/2026
=====
Select any item
and follow the hints. Have a nice day!
=====
Your choice:

```

Рисунок 2.6 – Вікно при від’ємній ціні.

```

Date must be in format DD/MM/YYYY
=====
Select product:
=====
1. Yogurt.
2. Milk.
3. Bread.
=====
Select operation:
=====
4. Expiry date check.
5. Sorting products on the shelf.
6. Buy products
7. Revenue from sold products .
8. Checking the product for integrity.
9. Products on the shelf.
10. Shelf capacity.
11. Exit from the program.
Today's date 27/05/2026
=====
Select any item
and follow the hints. Have a nice day!
=====
Your choice:

```

Рисунок 2.7 – Вікно при неправильному форматі дати

```

2
Calories cannot be negative.
=====
Select product:
=====
1. Yogurt.
2. Milk.
3. Bread.
=====
Select operation:
=====
4. Expiry date check.
5. Sorting products on the shelf.
6. Buy products
7. Revenue from sold products .
8. Checking the product for integrity.
9. Products on the shelf.
10. Shelf capacity.
11. Exit from the program.
Today's date 27/05/2026
=====
Select any item
and follow the hints. Have a nice day!
=====
Your choice:

```

Рисунок 2.8 – Вікно при від’ємних калоріях

```

2
You have indicated that the fat content in milk exceeds 4%
=====
Select product:
=====
1. Yogurt.
2. Milk.
3. Bread.
=====
Select operation:
=====
4. Expiry date check.
5. Sorting products on the shelf.
6. Buy products
7. Revenue from sold products .
8. Checking the product for integrity.
9. Products on the shelf.
10. Shelf capacity.
11. Exit from the program.
Today's date 27/05/2026
=====
Select any item
and follow the hints. Have a nice day!
=====
Your choice:

```

Рисунок 2.9 – Вікно при неправильному вводі жирності

Після вибору пункту 4 програма порівнює термін придатності кожного товару з поточною датою магазину. Прострочені товари видаляються з полиці, місце звільняється. На екран виводиться повідомлення про кількість знайдених прострочених позицій (Рис 2.11), або про те, що прострочених товарів немає (Рис 2.10).

```

Your choice:
4
Checking the shelf life of goods.
There are no expired products

```

Рисунок 2.10 – Результат перевірки терміну придатності

```

Your choice:
4
Checking the shelf life of goods.
Found expired products: 1

```

Рисунок 2.11 – Результат перевірки терміну придатності(коли термін придатності вийшов)

Користувачу пропонується обрати (Рис 2.12):

- а) сортування за зростанням ціни;
- б) сортування за спаданням ціни;
- с) вихід до головного меню.

Якщо на полиці немає товарів, програма повідомляє, що сортування неможливе (Рис 2.13).

```

Your choice:
5
Sorting products by price.
You can sort products by price.

You can sort products in ascending order by clicking - 1.
You can sort products in descending order by clicking - 2.
You can exit by clicking - 3.

Enter your choice:
1
Products are sorted (from cheap to expensive).

You can sort products in ascending order by clicking - 1.
You can sort products in descending order by clicking - 2.
You can exit by clicking - 3.

Enter your choice:
2
Products are sorted (from expensive to cheap).

You can sort products in ascending order by clicking - 1.
You can sort products in descending order by clicking - 2.
You can exit by clicking - 3.

Enter your choice:
3
You left case 5.

```

Рисунок 2.12 – Підменю сортування за ціною


```

There are no products on the shelf. Unable to sort.

You can sort products in ascending order by clicking - 1.
You can sort products in descending order by clicking - 2.
You can exit by clicking - 3.

```

Рисунок 2.13– Підменю сортування за ціною(коли немає продуктів)

Доступні варіанти:

- а) купити товар за назвою (потрібно ввести назву) (Рис 2.14);
- б) купити всі товари на полиці (Рис 2.15);
- с) повернення до головного меню (Рис 2.16);

При успішній покупці ціна товару додається до загальної виручки, товар зникає з полиці, місце звільняється (Рис 2.15). Якщо товар з такою назвою не знайдено, виводиться відповідне повідомлення (Рис 2.17).

```

Your choice:
6
In this subsection, you can purchase the product that is on the shelf.

You can buy a product by name by clicking - 1.
You can buy all products at once by clicking - 2.
You can exit by pressing - 3.

Enter your choice:
1
Enter product name:
DANONE

```

Рисунок 2.14 – Підменю оформлення покупки(за назвою)

```

Enter your choice:
2
You bought all the products.

You can buy a product by name by clicking - 1.
You can buy all products at once by clicking - 2.
You can exit by pressing - 3.

```

Рисунок 2.15 – Підменю оформлення покупки всіх товарів

```
Enter your choice:
3
You left case 6.
```

Рисунок 2.16 – Повернення до головного меню\

```
Enter your choice:
1
Enter product name:
thtj
Product 'thtj' not found.

You can buy a product by name by clicking - 1.
You can buy all products at once by clicking - 2.
You can exit by pressing - 3.
```

Рисунок 2.17 – Результат покупки при не правильному ввводі назви

Після вибору пункту 7 на екран виводиться сума виручки в гривнях (Рис2.18) або повідомлення про те, що жоден товар ще не було продано (Рис 2.19);

```
Your choice:
7
You sold products worth 2301 UAH
```

Рисунок 2.18 – Виведення суми виручки

```
Your choice:
7
You have not sold any products.
```

Рисунок 2.19 – Виведення того що жоден товар ще не було продано

Виведення підменю перевірки цілісності (Рис 2.20).

Користувач може:

1. перевірити цілісність товару за назвою (Рис 2.21);

2. перевірити всі товари на полиці (Рис 2.22);
3. повернутися до головного меню (Рис 2.23).

Якщо товар знайдено, програма повідомляє, що він цілий (Рис 2.21); якщо назва не збігається з жодним товаром на полиці — виводиться повідомлення про відсутність товару (Рис 2.24).

```
Your choice:
8
In this subsection, you can check products for integrity.

You can check the integrity of the product by name by clicking - 1.
You can check all products for integrity at once by clicking - 2.
You can exit by pressing - 3.

Enter your choice:
|
```

Рисунок 2.20 – Підменю перевірки цілісності

```
Enter your choice:
1
Enter the product name:
Yagotinske
Product 'Yagotinske' whole. No damage.

You can check the integrity of the product by name by clicking - 1.
You can check all products for integrity at once by clicking - 2.
You can exit by pressing - 3.
```

Рисунок 2.21 – Підменю перевірених товарів на цілісність за назвою

```
Enter your choice:
2
All products on the shelf are intact. No damage was found.

You can check the integrity of the product by name by clicking - 1.
You can check all products for integrity at once by clicking - 2.
You can exit by pressing - 3.
```

Рисунок 2.22 – Підменю перевірених всіх товарів на цілісність

```

You left case 8.
=====
Select product:
=====
1. Yogurt.
2. Milk.
3. Bread.
=====
Select operation:
=====
4. Expiry date check.
5. Sorting products on the shelf.
6. Buy products
7. Revenue from sold products .
8. Checking the product for integrity.
9. Products on the shelf.
10. Shelf capacity.
11. Exit from the program.
Today's date 27/05/2026
=====
Select any item
and follow the hints. Have a nice day!
=====
Your choice:

```

Рисунок 2.23 – Повернення до головного меню з перевірки на цілісність

```

Enter your choice:
1
Enter the product name:
DANONE
Product 'DANONE' not found.

You can check the integrity of the product by name by clicking - 1.
You can check all products for integrity at once by clicking - 2.
You can exit by pressing - 3.

```

Рисунок 2.24 – Випадок коли назва не збігається з жодним товаром на полиці

На екран виводиться повний перелік товарів із детальним описом кожного (метод Info()): назва, дати, вага, ціна, термін придатності, калорії та специфічні характеристики (жирність, начинка, тип упаковки, борошно тощо) (Рис 2.25). Якщо полиця порожня, виводиться відповідне повідомлення (Рис 2.25).

```

Your choice:
9
Products on the shelf:
Name: Yagotinske. Produced 27/05/2026 Weight: 579(g) Price 78$ Ends 20/06/2026 Calories : 568 call Fat: 5% Filler: straw
berry
Name: Yagotinske. Produced 27/05/2026 Weight: 579(g) Price 78$ Ends 20/06/2026 Calories : 568 call Fat: 5% Filler: straw
berry
Name: Yagotinske. Produced 27/05/2026 Weight: 579(g) Price 78$ Ends 20/06/2026 Calories : 568 call Fat: 5% Filler: straw
berry

```

Рисунок 2.25 – Список товарів на полиці

```

Your choice:
9
Products on the shelf:
There are no products on the shelf.

```

Рисунок 2.26 – Вивід при порожній полиці

Відкриття підменню керування місткістю полиці (Рис 2.27).

Користувач може:

- збільшити кількість місць на полиці (Рис 2.28);
- зменшити кількість місць (Рис 2.29) (якщо введені значення більше за поточну місткість, операція відхиляється) (Рис 2.30);
- переглянути поточну кількість вільних місць (Рис 2.31);
- повернутися до головного меню (Рис 2.32).

За замовчуванням полиця має 6 вільних місць.

```

Your choice:
10

You can increase the number of shelf spaces by clicking - 1.
You can reduce the number of shelf spaces by clicking - 2.
You can view the standard number of shelf slots by clicking - 3.
You can exit by clicking - 4.

```

Рисунок 2.27 – Підменю керування місткістю полиці

```

Your choice:
10

You can increase the number of shelf spaces by clicking - 1.
You can reduce the number of shelf spaces by clicking - 2.
You can view the standard number of shelf slots by clicking - 3.
You can exit by clicking - 4.

Enter your choice:
1
Enter gain:
2

```

Рисунок 2.28 – Підменю керування місткістю полиці при збільшити кількість місць на полиці

```

Enter your choice:
2
Enter gain:
2

You can increase the number of shelf spaces by clicking - 1.
You can reduce the number of shelf spaces by clicking - 2.
You can view the standard number of shelf slots by clicking - 3.
You can exit by clicking - 4.

```

Рисунок 2.29. – Підменю керування місткістю полиці при зменшити кількість місць на полиці

```

Enter your choice:
2
Enter gain:
3
You can't make shelf capacity less than zero.

```

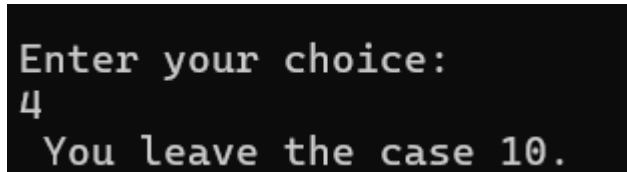
Рисунок 2.30 – Підменю керування місткістю полиці при зменшити кількість місць на полиці(при введенні більшого числа за поточну місткість)

```

Enter your choice:
3
Shelf seat now: 1.

```

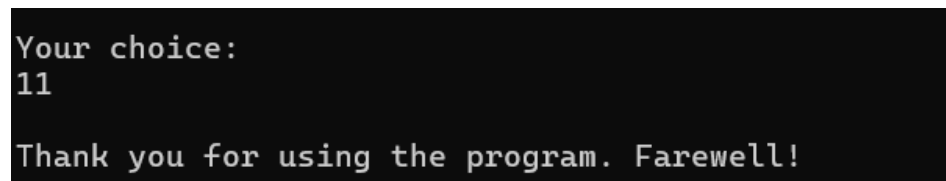
Рисунок 2.31 – Переглянути поточну кількість вільних місць



```
Enter your choice:  
4  
You leave the case 10.
```

Рисунок 2.32 – Повернення до головного меню після керуванні поліції

Після вибору пункту 11 на екран виводиться прощальне повідомлення, і робота програми завершується (Рис 2.33).



```
Your choice:  
11  
Thank you for using the program. Farewell!
```

Рисунок 2.33 – Вихід з програми

ВИСНОВКИ

Під час виконання курсової роботи я розробив програму на мові C++ на тему «Продуктовий магазин», застосовуючи підходи об'єктно-орієнтованого програмування. Було побудована ієрархія класів, що відображає структуру реального магазину. Кожен клас отримав власні поля та методи, а взаємодія між ними дозволяє моделювати основні процеси торгівлі.

При написанні програми активно використовувались принципи ООП. Інкапсуляція забезпечила захист даних усередині класів, наслідування дозволило побудувати логічну ієрархію від загального до конкретного, а поліморфізм – гнучко працювати з різними видами товарів через спільний інтерфейс. Програма підтримує декілька практичних сценаріїв: надходження нових товарів, їх розміщення по полицях, автоматичне списання прострочених продуктів та оформлення покупки. Також була додана можливість відстежувати виручку магазину – загальну суму коштів, отриманих від продажів.

Виконання цієї роботи допомогло краще зрозуміти, як працюють класи, колекції та механізми наслідування у C++, а також як застосовувати їх для вирішення реальних задач. Розроблена програма показує, що навіть відносно проста система може повноцінно імітувати роботу справжнього продуктового магазину.

СПИСОК ЛІТЕРАТУРИ

1. С. А. Ярошко, Основи об'єктно-орієнтованого програмування з ілюстраціями на Borland Pascal та Borland Pascal for Windows [Електронне видання] / Львів 1998. – Режим доступу: <https://ami.lnu.edu.ua/wp-content/uploads/2020/04/OOP.pdf>
2. Web Library, Основні поняття ООП. [Електронне видання] / Тернопіль 2026. – Режим доступу: <http://weblib.com.ua/blog/osnovni-ponyattya-oop>
3. W3School, C++ OOP (Object-Oriented Programming). Режим доступу: https://www.w3schools.com/cpp/cpp_oop.asp
4. Репозиторій GitHub на курсовий – Режим доступу: <https://github.com/darynanedieva-coder/Kursach>

Додаток А. Діаграма класів

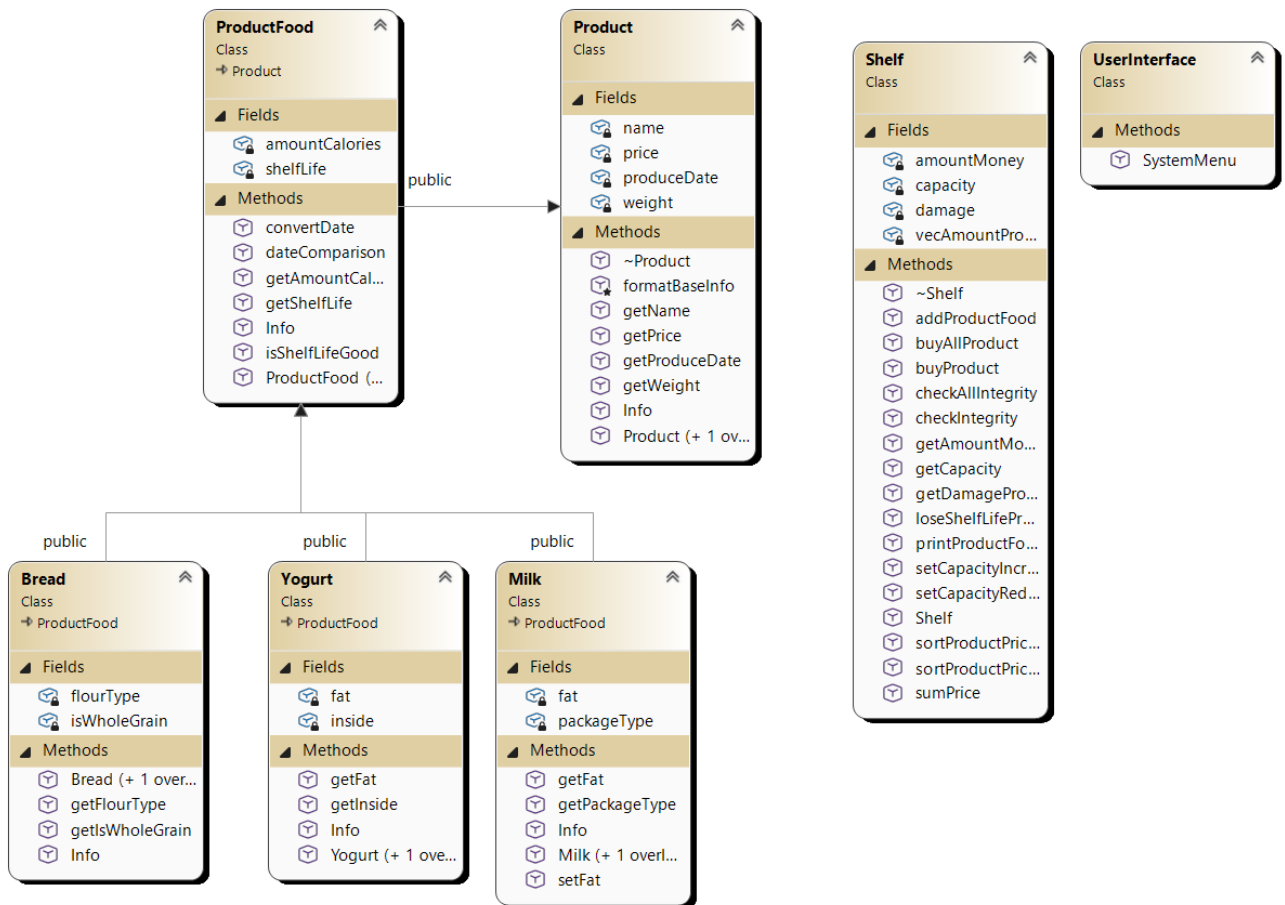


Рисунок А1. – Діаграма класів

Додаток Б. Код класу

```

#include "Yogurt.h"

Yogurt::Yogurt() : fat(6), inside("Strawberry") {}

Yogurt::Yogurt(const string& nameYogurt,
               const string& produceDate,
               int weight,
               int price,
               const string& shelfLife,
               int amountCalories,
               double fat,
               string inside)
    : ProductFood(nameYogurt, produceDate, weight, price, shelfLife, amountCalories),
      fat(fat),
      inside(inside) {

    // Перевірки на коректність даних
    if (weight < 0) {
        throw invalid_argument("The weight of yogurt cannot be negative.");
    }
    if (price < 0) {
        throw invalid_argument("The price of yogurt cannot be negative.");
    }
    if (amountCalories < 0) {
        throw invalid_argument("Several calories cannot be negative.");
    }
    if (fat < 0) {
        throw invalid_argument("Yogurt fat content cannot be negative.");
    }
    if (fat > 10) {
        throw invalid_argument("The fat content of yogurt cannot exceed 10%.");
    }
}

string Yogurt::Info() const {
    string result;
    result = ProductFood::Info() +
        " Fat: " + to_string(fat) + "%" +
        " Filler: " + inside;
    return result;
}

```

Лістинг Б.1 – Клас Yogurt

```

#include "Milk.h"
#include <stdexcept>
#include <string>
using namespace std;

Milk::Milk() :
    ProductFood(),
    fat(2.5),

```

Лістинг Б.2 – Клас Milk, аркуш 1

```

        packageType("bottle") {
    }
Milk::Milk(const string& nameMilk,
          const string& produceDateMilk,
          int weight,
          int price,
          const string& shelfLifeMilk,
          int amountCalories,
          double fat,
          string packageType) :
    ProductFood(nameMilk, produceDateMilk, weight, price, shelfLifeMilk, amountCalories),
    fat(fat),
    packageType(packageType)
{
    if (weight < 0) {
        throw invalid_argument("The weight of the milk cannot be negative.");
    }
    if (price < 0) {
        throw invalid_argument("The price of milk cannot be negative.");
    }
    if (amountCalories < 0) {
        throw invalid_argument("Calories cannot be negative.");
    }
    if (fat < 0) {
        throw invalid_argument("You have injected a milk fat content of less than 0%");
    }
    if (fat > 4) {
        throw invalid_argument("You have indicated that the fat content in milk exceeds
4%");
    }
}

void Milk::setFat(double newFat) {
    if (newFat < 0 || newFat > 4) {
        throw invalid_argument("The fat content should be between 0% and 4%");
    }
    fat = newFat;
}

string Milk::Info() const {
    string out;

    out = ProductFood::Info() +
        " Fat: " + to_string(getFat()) + "%" +
        " PackageType: " + getPackageType();

    return out;
}

```

Лістинг Б.2 – Клас Milk, аркуш 2

```

#include "Bread.h"
Bread::Bread() : flourType("wheat"), isWholeGrain(false) {}
Bread::Bread(const string& name,
             const string& produceDate,
             int weight,
             int price,

```

Лістинг Б.3 – Клас Bread, аркуш 1

```

const string& shelfLife,
int amountCalories,

string flourType,
bool isWholeGrain)
: ProductFood(name, produceDate, weight, price, shelfLife, amountCalories),
flourType(flourType),
isWholeGrain(isWholeGrain) {

    if (weight < 0) {
        throw invalid_argument("The weight of the bread cannot be negative.");
    }
    if (price < 0) {
        throw invalid_argument("The price of bread cannot be negative.");
    }
    if (amountCalories < 0) {
        throw invalid_argument("Calories cannot be negative.");
    }
}

string Bread::Info() const {
    string result;
    result = ProductFood::Info() +
        " Flour type: " + flourType +
        " Wholemeal: ";
    if (isWholeGrain)
        result = result + "Yes";
    else
        result = result + "No";
    return result;
}

```

Лістинг Б.3 – Клас Bread, аркуш 2

```

#include "ProductFood.h"

ProductFood::ProductFood() :
    shelfLife("DD/MM/YYYY"),
    amountCalories(500) {
}

ProductFood::ProductFood(const string& name,
    const string& produceDate,
    int weight,
    int price,
    const string& shelfLife,
    int amountCalories) :
    Product(name, produceDate, weight, price),
    shelfLife(shelfLife),
    amountCalories(amountCalories) {
    ProductFood::dateComparison(produceDate, shelfLife);
}

int ProductFood::convertDate(const std::string& date) {
    if (date.size() < 10 || date[2] != '/' || date[5] != '/') {
        throw std::invalid_argument("Date must be in format DD/MM/YYYY");
    }
    int day = (date[0] - '0') * 10 + (date[1] - '0');
    int month = (date[3] - '0') * 10 + (date[4] - '0');
}

```

Лістинг Б.4 – Клас ProductFood, аркуш 1

```

        int year = (date[6] - '0') * 1000 + (date[7] - '0') * 100 +
            (date[8] - '0') * 10 + (date[9] - '0');
        return year * 10000 + month * 100 + day;
    }
    bool ProductFood::dateComparison(const string& ProduceDate, const string& ShelfLife) {
        int produceDate = convertDate(ProduceDate);
        int shelfLife = convertDate(ShelfLife);
        if (produceDate > shelfLife) {
            throw invalid_argument("Production date cannot be later than the expiration
date.");
        }
        return true;
    }
    bool ProductFood::isShelfLifeGood(const string& currentDate) const {
        int shelfLifeDate = convertDate(getShelfLife());
        int currentDateNum = convertDate(currentDate);

        return shelfLifeDate >= currentDateNum;
    }

    string ProductFood::Info() const {
        string out;

        out = formatBaseInfo() +
            " Ends " + getShelfLife() +
            " Calories : " + to_string(getAmountCalories()) + " call";

        return out;
    }

```

Лістинг Б.4 – Клас ProductFood, аркуш 2

```

#include "Product.h"

Product::Product() :
    name("Normal"),
    produceDate("DD/MM/YYYY"),
    weight(0),
    price(0) {
}

Product::Product(const string& name,
    const string& produceDate,
    int weight,
    int price) :
    name(name),
    produceDate(produceDate),
    weight(weight),
    price(price) {
}

string Product::formatBaseInfo() const {
    return "Name: " + getName() + "." +
        " Produced " + getProduceDate() +
        " Weight: " + to_string(getWeight()) + "(g)" +
        " Price " + to_string(getPrice()) + " UAH";
}

```

Лістинг Б.5 – Клас Product

```

#include "Shelf.h"

using namespace std;

Shelf::Shelf() : capacity(6), damage(0), amountMoney(0) {}

Shelf::~Shelf() {
    for (int i = 0; i < vecAmountProduct.size(); i++) {
        delete vecAmountProduct[i];
    }
}

void Shelf::addProductFood(const ProductFood* product, int count) {
    if (count < 0) {
        throw invalid_argument("You cannot add a negative quantity.");
    }
    if (count > capacity) {
        throw invalid_argument("There is not enough space on the shelf.");
    }
    vecAmountProduct.push_back(product);
    capacity--;
}

// Виделення прострочених продуктів
void Shelf::loseShelfLifeProductFood(const string& date) {
    if (vecAmountProduct.size() == 0) {
        cout << "There are no products on the shelf" << endl;
        return;
    }
    damage = 0;
    for (int i = 0; i < vecAmountProduct.size(); i++) {
        if (vecAmountProduct[i]->isShelfLifeGood(date) == false) {
            delete vecAmountProduct[i];
            vecAmountProduct.erase(vecAmountProduct.begin() + i);
            capacity++;
            damage++;
            i--;
        }
    }
    if (damage > 0) {
        cout << "Found expired products: " << damage << endl;
    }
    else {
        cout << "There are no expired products" << endl;
    }
}

// СОРТУВАННЯ за зростанням ціни
void Shelf::sortProductPriceAscending() {
    if (vecAmountProduct.empty()) {
        cout << "There are no products on the shelf. Unable to sort." << endl;
        return;
    }
    int n = static_cast<int>(vecAmountProduct.size());
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {

```

```

        if (vecAmountProduct[j]->getPrice() > vecAmountProduct[j + 1]->getPrice()) {
            const ProductFood* temp = vecAmountProduct[j];
            vecAmountProduct[j] = vecAmountProduct[j + 1];
            vecAmountProduct[j + 1] = temp;
        }
    }
}
cout << "Products are sorted (from cheap to expensive)." << endl;
}

// СОРТУВАННЯ за спаданням ціни
void Shelf::sortProductPriceDescending() {
    if (vecAmountProduct.empty()) {
        cout << "There are no products on the shelf. Unable to sort." << endl;
        return;
    }
    int n = static_cast<int>(vecAmountProduct.size());
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (vecAmountProduct[j]->getPrice() < vecAmountProduct[j + 1]->getPrice()) {
                const ProductFood* temp = vecAmountProduct[j];
                vecAmountProduct[j] = vecAmountProduct[j + 1];
                vecAmountProduct[j + 1] = temp;
            }
        }
    }
    cout << "Products are sorted (from expensive to cheap)." << endl;
}

// Купівля одного продукту за назвою
void Shelf::buyProduct(const string& name) {
    if (vecAmountProduct.empty()) {
        cout << "There are no products on the shelf." << endl;
        return;
    }
    bool found = false;
    for (int i = 0; i < vecAmountProduct.size(); i++) {
        if (vecAmountProduct[i]->getName() == name) {
            sumPrice(vecAmountProduct[i]->getPrice());
            delete vecAmountProduct[i];
            vecAmountProduct.erase(vecAmountProduct.begin() + i);
            capacity++;
            found = true;
            break;
        }
    }
    if (found) {
        cout << "You bought " << name << "." << endl;
    }
    else {
        cout << "Product '" << name << "' not found." << endl;
    }
}

// Купити всі продукти
void Shelf::buyAllProduct() {
    if (vecAmountProduct.empty()) {
        cout << "There are no products on the shelf." << endl;
        return;
    }
}

```



```

    }
    for (int i = 0; i < vecAmountProduct.size(); ) {
        sumPrice(vecAmountProduct[i]->getPrice());
        delete vecAmountProduct[i];
        vecAmountProduct.erase(vecAmountProduct.begin() + i);
        capacity++;
    }
    cout << "You bought all the products." << endl;
}
// Перевірка цілісності одного продукту (без випадковості)
void Shelf::checkIntegrity(const string& name) {
    if (vecAmountProduct.empty()) {
        cout << "There are no products on the shelf." << endl;
        return;
    }
    bool found = false;
    for (int i = 0; i < vecAmountProduct.size(); i++) {
        if (vecAmountProduct[i]->getName() == name) {
            found = true;
            break;
        }
    }
    if (found) {
        cout << "Product '" << name << "' whole. No damage." << endl;
    }
    else {
        cout << "Product '" << name << "' not found." << endl;
    }
}

// Перевірка всіх продуктів (без випадковості)
void Shelf::checkAllIntegrity() {
    if (vecAmountProduct.empty()) {
        cout << "There are no products on the shelf." << endl;
        return;
    }
    cout << "All products on the shelf are intact. No damage was found." << endl;
}

// Вивести всі продукти
void Shelf::printProductFood() {
    if (vecAmountProduct.empty()) {
        cout << "There are no products on the shelf." << endl;
    }
    else {
        for (int i = 0; i < vecAmountProduct.size(); i++) {
            cout << vecAmountProduct[i]->Info() << endl;
        }
    }
}
}

```

Лістинг Б.6 – клас Shelf, аркуш 3

```

#include "UserInterface.h"

#include <cstdlib>
#include <iostream>

```

Лістинг Б.7 – Клас UserInterface, аркуш 1

```

void UserInterface::SystemMenu() {
    int weight, price, amountCalories, amount, gain, selection;
    double milkFat;
    double fatYogurt;
    string name, inside, packageType, produceDate, shelfLife, flourType;
    string CurrentDate = "27/05/2026";
    Yogurt* yogurt;
    Milk* milk;
    Bread* bread;
    Shelf shelf;
    cout << "|=====|\n"
         " |          WELCOME TO THE STORE          |\n";
    do {
        cout << "|=====|\n"
             " | Select product:                               |\n"
             " |=====|\n"
             " | 1. Yogurt.                                     |\n"
             " | 2. Milk.                                       |\n"
             " | 3. Bread.                                     |\n"
             " |=====|\n"
             " | Select operation:                             |\n"
             " |=====|\n"
             " | 4. Expiry date check.                         |\n"
             " | 5. Sorting products on the shelf.              |\n"
             " | 6. Buy products                               |\n"
             " | 7. Revenue from sold products .               |\n"
             " | 8. Checking the product for integrity.         |\n"
             " | 9. Products on the shelf.                     |\n"
             " | 10. Shelf capacity.                           |\n"
             " | 11. Exit from the program.                     |\n"
             " | Today's date "; cout << CurrentDate; cout << " |\n";
        cout << "|=====|\n"
             " | Select any item                               |\n"
             " | and follow the hints. Have a nice day!        |\n"
             " |=====|\n";

        cout << "\nYour choice:\n"; cin >> selection;
        switch (selection) {
            case 1:
                try {
                    cout << "You chose yogurt, let's add it to the shelf.\n";
                    cout << "Enter yogurt name:\n"; cin.ignore(); getline(cin, name);
                    cout << "Add the production date as (DD/MM/YYYY):\n"; getline(cin,
produceDate);
                    cout << "Enter weight in grams (not less than 0):\n"; cin >> weight;
                    cout << "Enter price UAH:\n"; cin >> price;
                    cout << "Enter the expiration date (DD/MM/YYYY):\n"; cin.ignore();
getline(cin, shelfLife);
                    cout << "Enter the number of calories (kcal) at least 0:\n"; cin >>
amountCalories;
                    cout << "Inject the fat content of the yogurt (0% to 10%) :\n"; cin >>
fatYogurt;
                    cout << "Inject the yogurt filling (strawberry, apple and other fruit):\n";
                    cout << "Enter the inside of the yogurt:\n"; cin.ignore(); getline(cin,
inside);
                    cout << "Free places on the shelf: " << shelf.getCapacity() << " .\n";
                    cout << "Product quantity:\n"; cin >> amount;

```

Лістинг Б.7 – Клас UserInterface, аркуш 2

```

        if (amount > shelf.getCapacity()) {
            cout << "There is not enough space on the shelf.\n";
            break;
        }

        for (int i = 0; i < amount; i++) {
            yogurt = new Yogurt(name, produceDate, weight, price, shelfLife,
amountCalories, fatYogurt, inside);
            shelf.addProductFood(yogurt, 1);
        }
        if (amount > 1) {
            cout << " " << amount << " yogurts have been added to the shelf.\n";
        }
        else {
            cout << " " << amount << " yogurt was added to the shelf.\n";
        }
    }
    catch (exception& ex) {
        cout << ex.what() << endl;
    }
    break;
case 2:
    try {
        cout << "You have selected milk, we will add it to the shelf.\n";
        cout << "Enter the name of the milk:\n"; cin.ignore(); getline(cin, name);
        cout << "Add the production date as (DD/MM/YYYY): \n"; getline(cin, produceDate);
        cout << "Enter weight in grams (not less than 0):\n"; cin >> weight;
        cout << "Enter price UAH:\n"; cin >> price;
        cout << "Enter the expiration date (DD/MM/YYYY):\n"; cin.ignore(); getline(cin,
shelfLife);
        cout << "Enter the number of calories (kcal) at least 0:\n"; cin >> amountCalories;
        cout << "Enter the fat content of the milk (from 0% to 4%):\n"; cin >> milkFat;
        cout << "Enter the type of packaging (Bottle, pure pack, pillow):\n"; cin.ignore();
getline(cin, packageType);
        cout << "Free places on the shelf: " << shelf.getCapacity() << ".\n";
        cout << "Product quantity:\n"; cin >> amount;

        if (amount > shelf.getCapacity()) {
            cout << "There is not enough space on the shelf.\n";
            break;
        }

        for (int i = 0; i < amount; i++) {
            milk = new Milk(name, produceDate, weight, price, shelfLife, amountCalories,
milkFat, packageType);
            shelf.addProductFood(milk, 1);
        }
        cout << " " << amount << " milk (" << packageType << ") were added to the
shelf.\n";
    }
    catch (exception& ex) {
        cout << ex.what() << endl;
    }
    break;
case 3:
    try {
        cout << "You chose bread, let's add it to the shelf.\n";
        cout << "Enter the name of the bread:\n"; cin.ignore(); getline(cin, name);

```

```

        cout << "Add the production date as (DD/MM/YYYY):\n"; getline(cin, produceDate);
        cout << "Enter weight in grams (not less than 0):\n"; cin >> weight;
        cout << "Enter price UAH:\n"; cin >> price;
        cout << "Enter the expiration date (DD/MM/YYYY):\n"; cin.ignore(); getline(cin,
shelfLife);
        cout << "Enter the number of calories (kcal) at least 0:\n"; cin >> amountCalories;
        cout << "Enter the type of flour (wheat, rye, buckwheat):\n"; cin.ignore();
getline(cin, flourType);
        cout << "Enter 1 if whole grain, if not 0.\n"; int wholeGrainInt;
        cin >> wholeGrainInt;
        bool isWholeGrain = (wholeGrainInt == 1);
        cout << "Free places on the shelf: " << shelf.getCapacity() << ".\n";
        cout << "product quantity:\n"; cin >> amount;

        if (amount > shelf.getCapacity()) {
            cout << "There is not enough space on the shelf.\n";
            break;
        }
        for (int i = 0; i < amount; i++) {
            bread = new Bread(name, produceDate, weight, price, shelfLife, amountCalories,
flourType, isWholeGrain);
            shelf.addProductFood(bread, 1);
        }
        cout << " Added " << amount << " bread loaf.\n";
    }
    catch (exception& ex) {
        cout << ex.what() << endl;
    }
    break;
case 4:
    cout << "Checking the shelf life of goods.\n";
    shelf.loseShelfLifeProductFood(CurrentDate);
    break;
case 5:
    cout << "Sorting products by price.\n"
        "You can sort products by price."; cout << endl;
    do {
        cout << "\nYou can sort products in ascending order by clicking - 1.\n"
            "You can sort products in descending order by clicking - 2.\n"
            "You can exit by clicking - 3.\n"
            "\nEnter your choice:\n"; cin >> selection;
        switch (selection) {
            case 1:
                shelf.sortProductPriceAscending();
                break;
            case 2:
                shelf.sortProductPriceDescending();
                break;
            case 3:
                cout << " You left case 5.\n";
                break;
            default:
                cout << "\n";
        }
    } while (selection != 3);
    break;
case 6:

```

```

cout << "In this subsection, you can purchase the product that is on the shelf.\n";
do {
    cout << "\nYou can buy a product by name by clicking - 1.\n"
        "You can buy all products at once by clicking - 2.\n"
        "You can exit by pressing - 3.\n"
        "\nEnter your choice:\n"; cin >> selection;
    switch (selection) {
        case 1:
            cout << "Enter product name:\n"; cin.ignore(); getline(cin, name);
            shelf.buyProduct(name);
            break;
        case 2:
            shelf.buyAllProduct();
            break;
        case 3:
            cout << " You left case 6.\n";
            break;
        default:
            cout << "\n";
    }
} while (selection != 3);
break;
case 7:
    if (shelf.getAmountMoney() == 0) {
        cout << " You have not sold any products." << "\n";
    }
    else {
        cout << " You sold products worth " << shelf.getAmountMoney() << " UAH\n";
    }
    break;
case 8:
    cout << "In this subsection, you can check products for integrity.\n";
    do {

        cout << "\nYou can check the integrity of the product by name by clicking - 1.\n"

        "You can check all products for integrity at once by clicking - 2.\n"
        "You can exit by pressing - 3.\n"
        "\nEnter your choice:\n"; cin >> selection;
        switch (selection) {
            case 1:
                cout << "Enter the product name:\n"; cin.ignore(); getline(cin, name);
                shelf.checkIntegrity(name);
                break;
            case 2:
                shelf.checkAllIntegrity();
                break;
            case 3:
                cout << " You left case 8.\n";
                break;
            default:
                cout << "\n";
        }
    } while (selection != 3);
    break;
case 9:

```

```

    cout << " Products on the shelf:\n";
    shelf.printProductFood();
    break;
case 10:
    do {
        cout << "\nYou can increase the number of shelf spaces by clicking - 1.\n"
            "You can reduce the number of shelf spaces by clicking - 2.\n"
            "You can view the standard number of shelf slots by clicking - 3.\n"
            "You can exit by clicking - 4.\n"
            "\nEnter your choice:\n"; cin >> selection;
        switch (selection) {
            case 1:
                cout << "Enter gain: " << "\n"; cin >> gain;
                shelf.setCapacityIncrease(gain);
                break;
            case 2:
                cout << "Enter gain: " << "\n"; cin >> gain;
                if (gain > shelf.getCapacity()) {
                    cout << "You can't make shelf capacity less than zero." << "\n";
                }
                else {
                    shelf.setCapacityReduce(gain);
                }
                break;
            case 3:
                cout << "Shelf seat now: " << shelf.getCapacity() << ".\n";
                break;
            case 4:
                cout << " You leave the case 10.\n";
                break;
            default:
                cout << "\n";
        }
    } while (selection != 4);
    break;
case 11:
    cout << "\nThank you for using the program. Farewell!\n";
    exit(0);
default:
    cout << "\nYou made the wrong choice.\n";
}
} while (selection != 0);
}

```

Лістинг Б.7 – Клас UserInterface, аркуш 6